

ZFS Reblock Worker

Reescritura segura de bloques ZFS a escala de producción

Versión 4.4.14

24 de septiembre de 2026

Autor: Hernán Dario Nacimiento

github.com/hnacimiento/zfs-reblock-worker

Repositorio y evidencia de pruebas: ver README.md (bilingüe), docs/testing/ y CHANGELOG.md

Resumen ejecutivo

ZFS Reblock Worker es un worker en Bash para TrueNAS SCALE / Linux^[25] que reescribe archivos grandes dentro de su propio dataset ZFS para que sus bloques adopten el recordsize que el dataset ya tiene configurado. Cambiar el recordsize de un dataset no convierte nada por sí solo: solo afecta escrituras nuevas. Este worker hace el trabajo real, archivo por archivo, sin arriesgar los datos.

Cada archivo se copia a un temporal en el mismo directorio, se verifica con un hash criptográfico contra el origen, y recién entonces reemplaza al original mediante un rename atómico dentro del mismo dataset — nunca se trunca ni se sobrescribe en el lugar. El proceso puede morir en cualquier instrucción, en cualquier momento, incluso por un corte de energía, y la siguiente ejecución reconstruye de forma segura qué estaba haciendo.

Estado actual: versión 4.4.14, con dieciséis rondas de pruebas —la enorme mayoría contra ZFS real, no simulado— y dos rebloqueos completos en producción real (Musicales y Videos/Series, 2.1 TiB, 238 archivos, 0 fallos). Sobrevivió dos reinicios reales de la máquina a mitad de copia sin perder ni un byte.

Por qué existe

Un administrador cambia el recordsize de un dataset de su valor por defecto a 1M. Ese cambio solo afecta a las escrituras nuevas^[1]: los archivos que ya están ahí conservan el tamaño de bloque con el que se escribieron. Para que el cambio sea real hay que reescribir los archivos grandes — y al reescribirlos no deben perder su identidad: mtime y atime tienen que conservar su timestamp original.

```
ORIGINAL --rsync -aAXS--> TEMPORAL (mismo directorio)
                        |
                    verificacion completa
                        |
                    rename atomico
                        |
                    ORIGINAL
```

Contrato de seguridad

Estas once propiedades son invariantes: no hay opción de línea de comandos que las desactive.

1. Inventario snapshot. Un job procesa exactamente el universo detectado al empezar.
2. Copia a un temporal en el mismo directorio, para garantizar que el commit es un rename dentro del mismo dataset.
3. Identidad del origen verificada (tamaño + inode + mtime) antes de copiar, durante, y otra vez justo antes del commit.
4. Hash criptográfico del origen contra el temporal antes del commit (default BLAKE2b^[17], cascada a BLAKE3^[18] y SHA-256^[19]).
5. atime y mtime capturados^[8] antes de leer el archivo y restaurados explícitamente.
6. Commit únicamente por rename.^[6] El original nunca se trunca ni se sobrescribe en el lugar.
7. Estado persistido antes y después de cada frontera relevante.
8. Recuperación conservadora: un temporal huérfano va a cuarentena, nunca se promueve ni se borra.
9. Ancla de rollback: un hardlink^[7] al inode original mantiene los datos alcanzables hasta que la verificación post-commit pasa.
10. El recordsize del dataset no se modifica jamás.
11. ACL y xattr verificados con la herramienta que exige el dataset; si no hay verificador, el archivo queda fuera del inventario.

Premisa de diseño: *el proceso puede morir en cualquier instrucción, en cualquier momento, y la próxima ejecución debe poder reconstruir de forma segura qué estaba haciendo y continuar.*

Qué tan probado está

Dieciséis rondas de pruebas, la enorme mayoría contra ZFS real sobre el mismo TrueNAS de referencia:

- Dos reinicios reales de la máquina a mitad de copia (uno planeado, uno por un corte eléctrico real) — el archivo original quedó intacto en ambos casos.
- kill -9, dos workers reales simultáneos sobre el mismo job, y Ctrl+C real — todos sin comprometer el original.
- Musicales completo y Videos/Series completo reblockeados en producción real: 2.1 TiB, 238 archivos, 0 fallos, verificado con hash, ACL, mtime y zdb.
- Una sesión completa de consola interactiva en vivo encontró y corrigió tres bugs reales; la ronda de producción a escala encontró y corrigió cinco más — ninguno de los ocho comprometió la integridad del original.

El detalle completo, ronda por ronda, con la evidencia cruda, está en docs/testing/ del repositorio. Lo que todavía no se probó: el resto del dataset (una decisión de rollout, no un test) y ACL POSIX no trivial / atime=on sobre un dataset de producción real (solo verificado en laboratorio).

Adaptive Resource Governor

Antes de tocar el primer archivo, el worker observa el servidor unos segundos y establece un baseline. Mientras trabaja, calcula un puntaje de salud a partir de CPU, I/O wait, load y memoria, y de cuánto se desvía todo eso del baseline. El I/O wait pesa más que la CPU, porque en un reblock el recurso delicado es el almacenamiento.

Tres mejoras recientes, encontradas y corregidas contra hardware real:

- La lectura de memoria descuenta el ARC de ZFS^[2] que sería reclamable sin problema — sin este ajuste, un host sano con ARC alto se leía como si tuviera un problema real.
- Después de una pausa protectiva, el governor espera un enfriamiento corto antes de volver a subir el ancho de banda, para no chocar con la siguiente ráfaga periódica de sincronización de ZFS.^[5]
- El puntaje de presión le da crédito a la evidencia de que el disco sigue entregando bytes reales — el mismo principio que usan TCP BBR^[15] y los limitadores de concurrencia adaptativa^[16]: un disco ocupado siendo útil no es lo mismo que uno ocupado y atascado.

Con --adaptive off el governor deja de actuar pero sigue midiendo; la autoridad del operador es siempre absoluta: un techo de BW manual nunca puede ser superado por el sistema automático.

Uso básico

Wizard, sin argumentos

```
./zfs-reblock-worker.sh
```

Detecta el estado de trabajos previos y solo ofrece las opciones que tienen sentido en esa situación. Incluye explorador de directorios.

Línea de comandos, para automatización

```
./zfs-reblock-worker.sh /mnt/toolbox/media --bwlmit 20M --adaptive guarded
./zfs-reblock-worker.sh /mnt/toolbox/media --dry-run --dry-sample 5
```

```
./zfs-reblock-worker.sh --check
./zfs-reblock-worker.sh --status --json
```

Ver `--help` en el propio script para la lista completa de opciones.

Dependencias

Tipo	Herramientas	Si falta
Críticas	bash awk find sort grep sed rsync ^[24] flock ^[9] zfs sync sha256sum id kill, más UID 0	El worker no arranca
Según <code>acltype</code> ^[3]	nfs4xdr_getfacl+jq (nfsv4) / getfacl+getfattr (posix) / getfattr (off)	Ese dataset queda fuera del inventario
Opcionales	zdb ^[4] nice ^[12] renice ionice ^[11] whiptail dialog fzf b2sum b3sum timeout	Degradación controlada, avisada, nunca fallo silencioso

Códigos de salida

Código	Significado
0	Todo correcto, o job pausado de forma segura
12	El job terminó con al menos un archivo en FAILED
13	El job se detuvo antes de tiempo (espacio, operador, governor)
20	Falta una dependencia crítica, o ningún dataset es verificable
23	El recordsize del dataset no es el esperado

Origen del proyecto

El worker actual es el resultado de dieciséis rondas de pruebas y correcciones sobre un diseño que arrancó, en septiembre de 2026, como una discusión de arquitectura punto por punto a partir de un prototipo inicial (V2) que solo verificaba tamaño y no tenía un checkpoint real. Ese historial completo — el prototipo original, las versiones intermedias y la discusión de diseño que originó el V3 — se conserva en la carpeta `legacy/` del repositorio, fuera del árbol principal, como registro honesto de dónde vino el proyecto sin mezclarlo con el estado actual.

Identidad y verificación de este documento

Este documento se publica firmado digitalmente (PAdES)^[22] con el certificado personal^[21] de Hernán Dario Nacimiento, publicado en `certificate/` del repositorio junto a su huella (fingerprint) SHA-256. La versión `.docx/.htm` se acompaña además de un archivo `.sha256` propio. Cualquier copia cuya firma no valide, o cuyo hash no coincida, no es el original. La fuente de verdad definitiva es el repositorio de GitHub del proyecto: cada versión queda anclada a un commit con su propio hash, verificable de forma independiente.

Referencias

1. [↑](#) — [zfsprops.7 — OpenZFS documentation](#). [Versión archivada](#) · Consultado el 24-09-2026.
2. [↑](#) — [Module Parameters — OpenZFS documentation \(zfs arc min\)](#). [Versión archivada](#) · Consultado el 24-09-2026.
3. [↑](#) — [zfsprops.7 — OpenZFS documentation \(acltype\)](#). [Versión archivada](#) · Consultado el 24-09-2026.
4. [↑](#) — [zdb.8 — OpenZFS documentation](#). [Versión archivada](#) · Consultado el 24-09-2026.
5. [↑](#) — [Module Parameters — OpenZFS documentation \(zfs txg timeout\)](#). [Versión archivada](#) · Consultado el 24-09-2026.
6. [↑](#) — [rename\(2\) — Linux manual page](#). [Versión archivada](#) · Consultado el 24-09-2026.
7. [↑](#) — [link\(2\) — Linux manual page](#). [Versión archivada](#) · Consultado el 24-09-2026.
8. [↑](#) — [zfsprops.7 — OpenZFS documentation \(atime\)](#). [Versión archivada](#) · Consultado el 24-09-2026.
9. [↑](#) — [flock\(2\) — Linux manual page](#). [Versión archivada](#) · Consultado el 24-09-2026.
11. [↑](#) — [ionice\(1\) — Linux manual page](#). [Versión archivada](#) · Consultado el 24-09-2026.
12. [↑](#) — [sched\(7\) — Linux manual page \(nice value\)](#). [Versión archivada](#) · Consultado el 24-09-2026.
15. [↑](#) — [TCP BBR congestion control comes to GCP \(Google Cloud Blog\)](#). [Versión archivada](#) · Consultado el 24-09-2026.
16. [↑](#) — [Adaptive Concurrency — Envoy documentation](#). [Versión archivada](#) · Consultado el 24-09-2026.
17. [↑](#) — [RFC 7693: The BLAKE2 Cryptographic Hash and MAC](#). [Versión archivada](#) · Consultado el 24-09-2026.
18. [↑](#) — [BLAKE3 — official repository \(BLAKE3-team\)](#). [Versión archivada](#) · Consultado el 24-09-2026.
19. [↑](#) — [FIPS PUB 180-4: Secure Hash Standard \(NIST\)](#). [Versión archivada](#) · Consultado el 24-09-2026.
21. [↑](#) — [RFC 5280: Internet X.509 PKI Certificate and CRL Profile](#). [Versión archivada](#) · Consultado el 24-09-2026.
22. [↑](#) — [ETSI EN 319 142-1: PAdES digital signatures](#). [Versión archivada](#) · Consultado el 24-09-2026.
24. [↑](#) — [rsync\(1\) man page \(rsync / Samba project\)](#). [Versión archivada](#) · Consultado el 24-09-2026.
25. [↑](#) — [First Official RELEASE of TrueNAS on Linux! \(iXsystems\)](#). [Versión archivada](#) · Consultado el 24-09-2026.

© Hernán Dario Nacimiento. Documento generado a partir del estado real y verificado del proyecto a la fecha indicada en la portada.